Part A

- **A1 - What is the difference between response.status_code and len(response.content)?**
  - response.status_code shows the HTTP response status, such as 200 for a successful request or 404 for a missing resource. len(response.content) shows the size of the response body in bytes.
- **A2 - Explain what slicing ([:50]) does.**
  - [:50] returns the first 50 characters of the response text. It limits the output without changing the actual response.
- **Practice A - Run the example again using the search word `banana`. Which part of the request changed? Did the status code change? Do not assume that a different response length indicates a vulnerability.**
  - The query parameter changed from q=apple to q=banana. Both requests returned HTTP 200. However, the response lengths differed because the searches returned different content.
- **Practice B - In one or two sentences, compare the GET and POST results. Why does a `405` response not necessarily indicate a vulnerability?**
  - GET retrieves information, while POST submits data. Both returned HTTP 200, although the POST response was HTML rather than the JSON returned by GET. A 405 response would indicate that the HTTP method is not allowed, which does not necessarily indicate a security vulnerability.

Part B

- **Practice C - Add `lemon` to the list and run the cell again. How many times does the loop run?**
  - The loop runs four times because the list contains four search terms: apple, banana, orange, and lemon.
- **Practice D - Add a `content_type` key to `observation` using `r.headers.get("Content-Type")`. Print that value.**
  - The content_type key stores the HTTP response's Content-Type header. For the search request, the observed value was application/json; charset=utf-8.
- **Practice E - Add `response_length` to each dictionary. Explain why a list of dictionaries is helpful for generating a report.**
  - A list of dictionaries organizes each HTTP request into a separate record with consistent fields. This makes it easier to compare results and export them into a CSV file.
- **Practice F - Call `get_status("banana")`. Then explain what `search_word` means inside the function. You will decide later whether your scanner needs a function that returns a whole dictionary instead of a single number.**
  - search_word is a function parameter representing the search term passed into get_status(). When calling get_status("banana"), the parameter receives the value "banana".
- **Practice G - Explain why a scanner should include a timeout and handle `requests.RequestException`.**
  - A timeout prevents the scanner from waiting indefinitely for a response. Handling requests.RequestException allows the program to report connection errors without crashing.

- **Practice H - Test the non-executable HTML string `<u>fsct8561-test</u>` as a search term. Does it appear unchanged? If so, does that prove XSS? Explain why or why not. Keep the tests limited to the local lab instance.**
    - Whether the HTML input appears unchanged depends on the actual response. Even if it is reflected, this does not confirm XSS because the application may escape the input, return it as JSON, or prevent it from executing in the browser.
- **Practice I - Explain what you observed. Why is comparing the probe response to a normal baseline more informative than looking at the probe alone?**
    - Comparing a normal request with a SQL syntax probe helps identify unusual behavior, such as different status codes or database error messages. These differences may indicate an issue but do not confirm SQL injection.
- **Practice J - Suggest two additional database-error phrases that a detector might check. Explain why a matching phrase alone cannot establish successful SQL injection.**
    - Two additional phrases are SQL syntax and ORA-. A matching error message may reveal database information, but it does not prove that the input successfully changed or executed a database query.
- **Practice K - Look at the `Server` response header using `.get("Server")`. What information, if any, is revealed? Why is the presence of a `Server` header not automatically a critical vulnerability?**
    - In my HTTP scanner output, the Server header was not present. When provided, this header can reveal information about the web server software. Its presence is not automatically critical because it does not demonstrate that an attacker can exploit the server.
- **Practice L - Why does the list contain only two alerts even though the loop processes three messages? In your final detector, decide what makes two alerts the "same" finding (for example, category + endpoint + observation).**
    - One alert was repeated, and the set prevented it from being added again. This avoids duplicate findings and makes the report easier to review.
- **Practice M - Open `practice_records.csv` and identify its column headings and two rows. Why is CSV useful for the final lab report?**
    - The headings are endpoint and status. The two sample rows are / with status 200 and /example with status 404. CSV files organize results into rows and columns, making them easy to open in Excel, compare, and include as evidence in the lab report.

Task 3 - Server and configuration analysis

The HTTP scanner collected response information from the local OWASP Juice Shop application. The homepage returned HTTP 200 with a Content-Type of `text/html; charset=UTF-8`. The GET search requests also returned HTTP 200, with a Content-Type of `application/json; charset=utf-8`.

The Server header was not present in the recorded responses. This means the server did not directly disclose its software information through that header. However, the absence of the header does not guarantee that implementation information cannot be discovered through other methods.

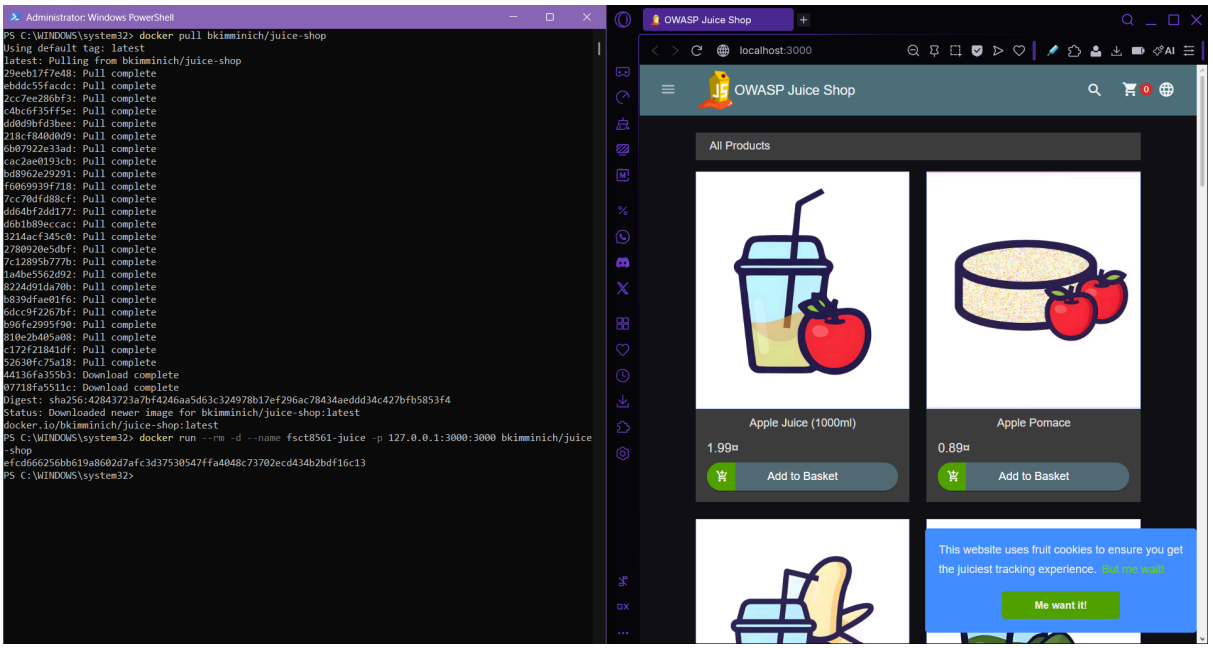
The vulnerability detector checks for the Content-Security-Policy and X-Content-Type-Options headers. Content-Security-Policy helps restrict where executable content can originate, while X-Content-Type-Options helps prevent MIME-type sniffing. Missing headers should be investigated because they may weaken browser protections, but their absence does not automatically establish an exploitable vulnerability.

HTTPS and TLS security cannot be properly evaluated through the local HTTP endpoint because the application is accessed through `http://localhost:3000`. In a real HTTPS deployment, I would examine the TLS version, certificate validity, supported cipher suites, and HTTPS configuration.

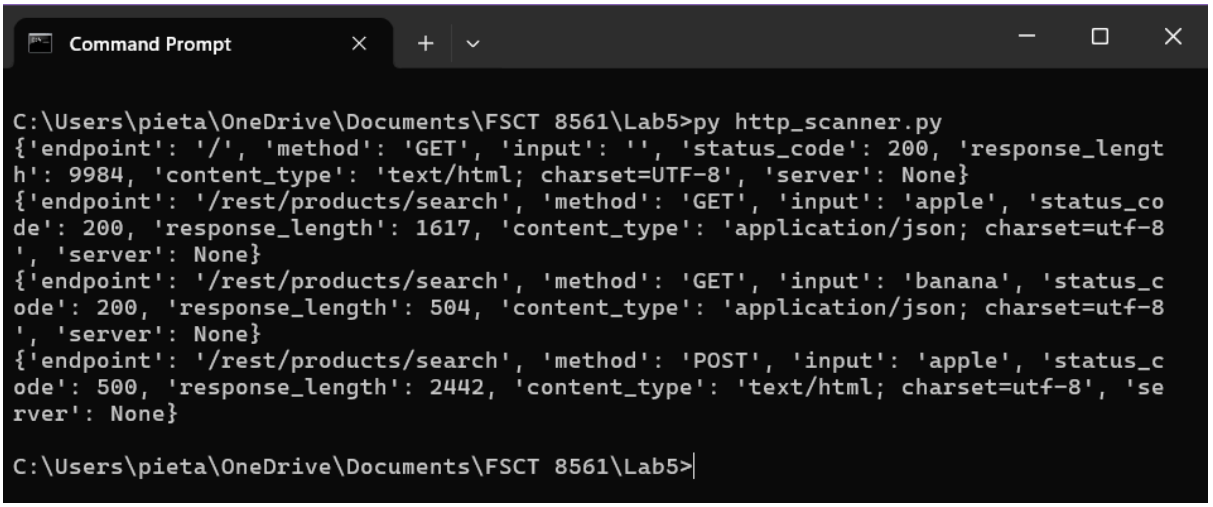
Finally, an HTTP 500 error or missing security header should be treated as a potential indicator rather than a confirmed vulnerability. Additional testing and manual verification are necessary to determine whether the observed behavior creates an exploitable security risk.

Task 4 - Evidence and Reproducibility

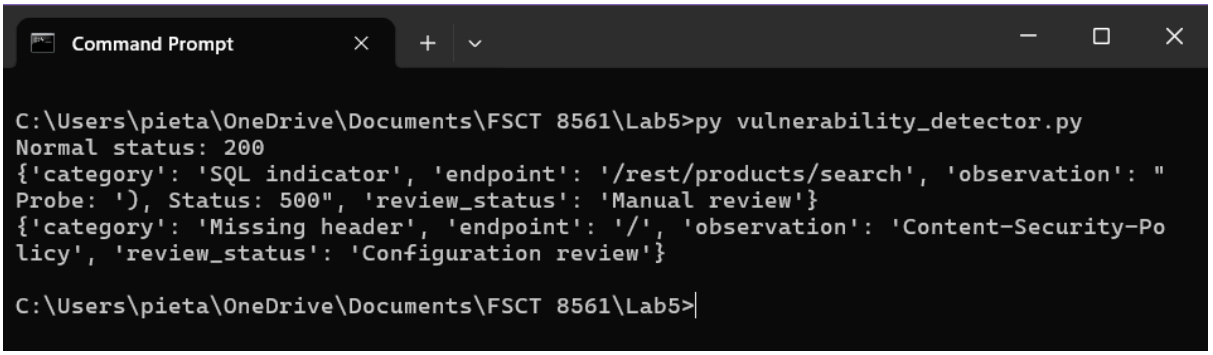
Your Juice Shop instance running locally



Successful execution of both Python scripts
http_scanner.py



vulnerability_detector.py



Task 5 - Reflection and security analysis

The HTTP scanner tested the local OWASP Juice Shop homepage and product search endpoint using GET and POST requests. All four recorded requests returned HTTP 200. The homepage returned HTML, while the GET search requests returned JSON. Response lengths differed across requests, showing that the application returned different content. The Server header was not present in the recorded scanner output. These observations demonstrate how the application responded, but they do not establish whether it is secure.

The vulnerability detector was designed to check for reflected input, SQL injection indicators, and missing security headers. Its actual findings must be interpreted using the generated `vulnerability_findings.csv` file. A reflected marker does not prove cross-site scripting because the text might be escaped or returned in JSON rather than executed. Likewise, an HTTP 500 response or database error is only a reason to investigate possible SQL injection, not evidence of successful exploitation. Missing security headers are configuration concerns whose significance depends on how the application is deployed.

As the security lead, I would prioritize manually reviewing any database-related errors or potentially unsafe reflected input found by the detector. I would then review missing browser security headers and determine whether additional protections are appropriate. This order

focuses first on behaviors that could potentially affect application data or users, while avoiding unsupported claims about confirmed vulnerabilities.

Lab 4 focused on passive packet analysis, where network traffic was observed without directly testing application inputs. Lab 5 extends that work into application-layer testing by using Python to send specific HTTP requests, examine responses, and record potential security indicators.

If Juice Shop used HTTPS, a normal network packet capture could still reveal connection metadata, such as IP addresses, ports, and some TLS handshake information, but not the encrypted HTTP request and response bodies. An authorized Python HTTP client could access the decrypted HTTP content because it participates in the TLS connection.

Automated findings require manual verification because status codes, error messages, and reflected strings can have harmless explanations. Verification helps establish the context, reproducibility, and actual security impact before a finding is reported as a confirmed vulnerability.